

KISANSETU
Offline Community Edge AI Hub for Indian Agriculture
Snapdragon® AI Lab Build & Present Challenge

Preserved	Strengthened
KisanSetu name and agricultural theme	Every identified requirement is mapped to a concrete section, test or evidence artifact.
One Snapdragon hub + many farmer smartphones	Snapdragon-specific proof strategy and CPU/NPU comparison
Offline multimodal AI	Exact model/runtime choices, APIs, scheduling and failure behavior
Vision + voice + local LLM + RAG + TTS	Safety gates, provenance, calibration and field validation
Community/FPO deployment	Economics, adoption, operations, maintenance and multi-hub lifecycle
Existing 20-section organization	Appendices expanded; no page-count limit

Development status: KisanSetu is specified as a build-and-validate system. Performance, accuracy, concurrency, power, cost and field-validation figures are recorded only after execution; no unmeasured result is presented as achieved.

1. Executive Summary

KisanSetu is an offline-first community agricultural AI service point. A Snapdragon-powered edge computer hosts the compute-intensive AI while several farmers use their existing smartphones as camera, microphone and display endpoints over local Wi-Fi. The hub is intended for FPOs, cooperatives, community agriculture centers and extension environments where continuous Internet access and individual AI hardware cannot be assumed.

The system combines crop-image triage, multilingual speech recognition, local language-model reasoning, local speech synthesis and a versioned agricultural knowledge base. A confidence/provenance/risk layer controls how the AI may respond. The system is assistive: it does not present uncertain visual classifications or unsupported chemical advice as authoritative.

The proposal explicitly does not claim novelty for crop disease classification, multilingual STT/TTS, local LLMs, RAG, offline operation or Snapdragon NPU usage individually. The architectural thesis is the shared community hub: one local Snapdragon compute resource can serve multiple ordinary smartphones.

The report converts each required proof point into a build artifact, acceptance test, evidence location and judge-facing explanation. The engineering specification and the evidence package are complementary: the report defines what must work and the evidence records what has actually been executed.

1.1 Evidence language — mandatory

Label	Use only when
DESIGNED	Architecture, workflow or policy is specified.
TARGET	A planned engineering value or target stack is stated.
MEASURED	An actual reproducible test produced the number.
DEMONSTRATED	The feature was actually shown running on the target hardware.
CALCULATED	The value follows from explicit recorded assumptions.
REFERENCE	The statement belongs to an external source.

Evidence language is used consistently so that design targets, measured results and demonstrated functions are never confused.

1.2 Final submission principle

The final package must contain both this report and an evidence bundle. The report explains the engineering; the evidence bundle proves execution. Neither part should be represented as the other.

2. Problem, Users and Requirements

2.1 Problem

- Farmers may possess smartphones without reliable Internet access for cloud AI.
- Crop-health assistance is inherently visual and benefits from voice interaction.
- Individual AI-capable hardware for every farmer is economically inefficient for community settings.
- Cloud-dependent workflows expose users to connectivity, privacy and recurring-service constraints.
- Incorrect diagnosis or unsafe chemical guidance can cause financial, agronomic and safety consequences.

2.2 Users

User	Primary job	Design requirement
Farmer	Ask, photograph, listen	One-tap camera/voice; clear confidence state.
FPO/cooperative	Serve many members	Shared hub, queueing, user isolation.
Operator	Keep hub healthy	Status, update, rollback, incident workflow.
Expert/KVK	Resolve uncertain cases	Escalation package with evidence and model version.

2.3 Requirements traceability

ID	Requirement	Verification
R01	Core AI works without WAN	OFF-01 to OFF-03
R02	≥ 3 simultaneous phones	CON-01
R03	5-client stress benchmark	CON-02
R04	Snapdragon/QNN path	NPU-01/NPU-02
R05	Field vision validation	VAL-01 to VAL-08
R06	Language validation	STT-01 to STT-06
R07	No unsupported high-risk advice	SAFE-01 to SAFE-10
R08	Signed updates + rollback	UPD-01 to UPD-04
R09	Privacy controls	SEC-01 to SEC-08
R10	Cost per farmer/query	ECON-01 to ECON-05

3. Prior Art and Existence Correction

The prior-art landscape shows that crop disease classification, multilingual STT/TTS, local LLMs, RAG, offline operation and Snapdragon NPU usage each have existing examples. KisanSetu therefore does not claim these individual capabilities as novel. Its architectural thesis is the shared community hub: one local Snapdragon compute resource can serve multiple ordinary smartphones.

Capability	Prior-art landscape	KisanSetu position
Crop disease AI	Plantix, KrishiAI, Traction, others	Baseline capability; not claimed novel.
Multilingual voice	A.A.H.A.R., AgriVerse-AI, KrishiVaani, GramGPT	Baseline capability; validate selected languages.
Local LLM	A.A.H.A.R., Arduino, Traction	Baseline capability; optimize for hub workload.
RAG	A.A.H.A.R., Farmsaaathi, AgriIR	Baseline capability; add provenance/safety governance.
Offline	Multiple prior systems	Requirement, not novelty.
Snapdragon NPU	Arduino, Traction, Qualcomm examples	Platform advantage to measure, not novelty by itself.
Community agricultural hub	No directly equivalent deployment model identified in the reviewed landscape	Primary architectural differentiator; requires demonstration.

3.1 Corrected positioning

KisanSetu is not positioned as 'the first agricultural AI'. It is positioned as a community deployment architecture that concentrates AI compute in a local Snapdragon hub and shares it among multiple existing smartphones.

3.2 Competitive honesty

The community-hub concept becomes a validated differentiator only when its operation is demonstrated. Terms such as “proven,” “deployed,” “field-validated” and “production-ready” are reserved for functions supported by corresponding artifacts and test records.

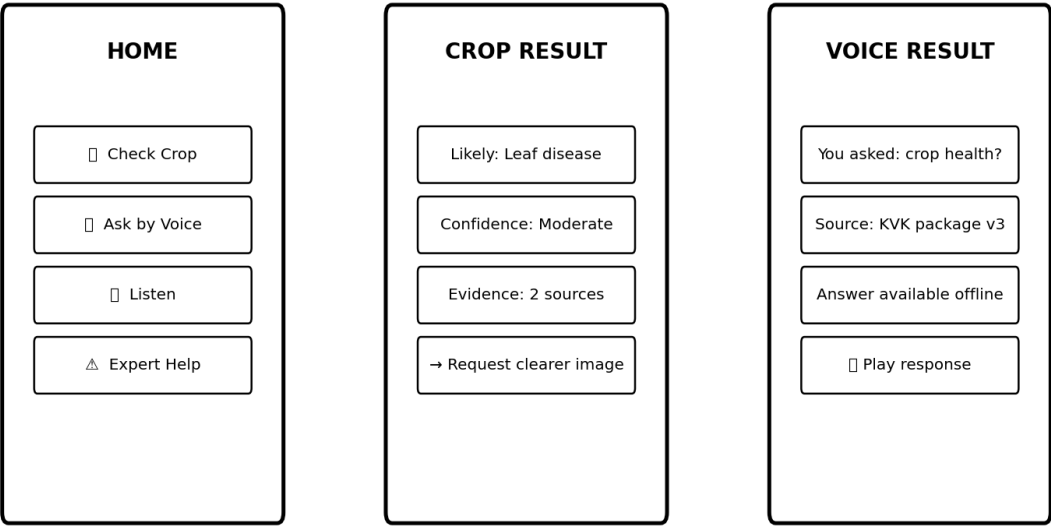
4. Solution and User Experience

4.1 Farmer workflow

1. Connect phone to the KisanSetu local Wi-Fi.
2. Open the local web application.
3. Select Camera, Speak or Text.
4. Receive a local response with confidence and evidence status.
5. Listen to the response or view it in text.
6. Request clarification or expert escalation when uncertain.

4.2 UI prototype

Figure 1. Designed UI mockups. These are design artifacts, not screenshots of a running build. A final submission should replace them with actual screenshots once the interface is implemented.

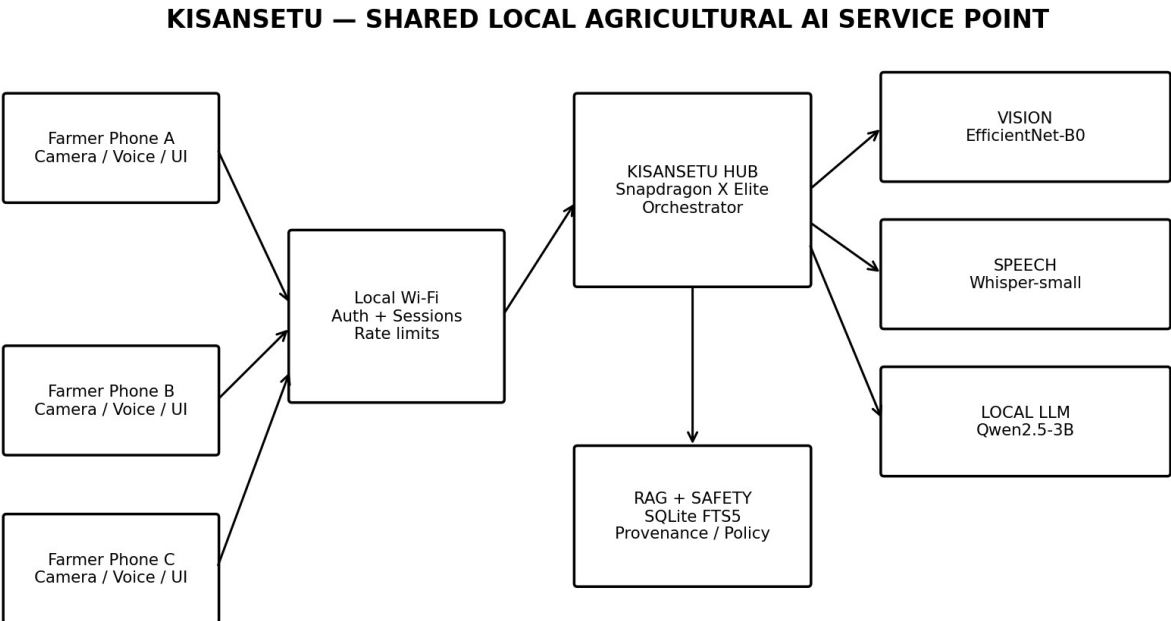


4.3 Accessibility

Barrier	Design response	Validation
Cannot type	Voice-first input	Farmer usability test.
Low literacy	Icons + spoken feedback	Task completion rate.
Poor camera quality	Image-quality prompt	Controlled blur/lighting test.
Uncertain answer	Confidence + next action	Safety usability test.
Language mismatch	Supported-language selector + fallback notice	Per-language evaluation.

5. System Architecture

Figure 2. Final KisanSetu architecture.



5.1 Components

Layer	Concrete implementation	Verification
Client	Responsive local web/PWA	Three phone browser test.
Gateway	FastAPI service	API integration tests.
Session manager	Tokenized sessions + quotas	Authorization tests.
Scheduler	Async bounded queues	Concurrency test.
Vision	EfficientNet-B0 baseline	NPU + accuracy tests.
STT	Whisper-small baseline	WER/latency tests.
LLM	Qwen2.5-3B-Instruct INT4 target	Runtime/memory tests.
RAG	SQLite FTS5 + metadata	Retrieval precision tests.
TTS	Piper/compatible local engine	Latency/intelligibility tests.
Safety	Confidence + provenance + risk policy	Red-team tests.
Update manager	Signed versioned packages	Rollback tests.

5.2 API contracts

Endpoint	Request	Response
POST /api/v1/session	device + language	session token/version
POST /api/v1/vision	image + session	classes/confidence/evidence
POST /api/v1/speech	audio + language	transcript/confidence
POST /api/v1/advice	query + context	grounded answer/sources/risk
GET /api/v1/status	token	service/model/resource status
GET /api/v1/knowledge	query/filter	source records/version

6. Snapdragon Implementation — Exact Execution Plan

Snapdragon is treated as a measurable systems dependency rather than a branding element. The implementation must prove model execution, resource behavior and the benefit of the chosen execution path on the actual device.

6.1 Target platform

Item	Target / record
Device	Exact Snapdragon X Elite SKU, OEM model and RAM recorded before benchmark.
Runtime	ARM64 environment with pinned ONNX Runtime/QNN versions.
Vision	EfficientNet-B0 INT8 candidate.
STT	Whisper-small baseline; smaller fallback if required.
LLM	Qwen2.5-3B-Instruct INT4 candidate.
NPU path	ONNX Runtime QNN Execution Provider / supported Qualcomm path.
Network	Local Wi-Fi access point.

6.2 AI Hub/QNN execution proof

7. Create the AI Hub job/profile for the exact target device.
8. Record model version, input shapes and quantization configuration.
9. Export the model artifact and checksum.
10. Run compatibility analysis and identify unsupported operators.
11. Capture execution-provider configuration.
12. Capture profiling output showing compute-unit placement.
13. Run the identical workload through the local hub.
14. Archive raw profiling logs and screenshots.

6.3 Snapdragon-specific benchmark

Run	Condition	Required output
S1	CPU-only	Latency, CPU, RAM, power.
S2	QNN/NPU	Latency, NPU, CPU, RAM, power.
S3	3 clients + QNN	P50/P95, queue, NPU/CPU.
S4	5 clients + QNN	P50/P95, queue, NPU/CPU, thermal.

The proposal should only claim a Snapdragon-specific advantage if S2 improves the relevant KisanSetu service metric versus the controlled S1 baseline. If the difference is small, report the result honestly and explain the other reasons the platform remains suitable.

Snapdragon implementation is treated as an engineering requirement rather than a branding claim. The execution path specifies Qualcomm AI Hub model selection/profiling, ONNX conversion, ONNX Runtime with QNN Execution Provider, Hexagon NPU execution, and a controlled CPU-versus-NPU comparison.

7. Vision AI Quality and Field Validation

7.1 Validation dataset

Split	Purpose	Rule
Train	Model fitting	Never used for final metrics.
Validation	Threshold/calibration selection	Freeze before field test.
Held-out test	Final laboratory benchmark	No tuning after evaluation.
Field set	Real-world robustness	Independent acquisition.

7.2 Required metrics

Metric	Report
Precision	Overall + per class
Recall	Overall + per class
F1	Macro + weighted
Specificity	Per class where meaningful
Confusion matrix	All supported classes
Calibration	Reliability curve / calibration error
False negative rate	Especially high-confidence subset
Image rejection rate	Unsupported/poor-quality inputs

7.3 Field protocol

15. Define supported crop/disease classes before collecting the final test set.
16. Use independent field images across lighting, devices, angles and backgrounds.
17. Use expert/reference labels where available.
18. Freeze model and thresholds before final evaluation.
19. Record model version, confidence and quality flags.
20. Analyze every high-impact false negative.
21. Report field and laboratory results separately.
22. Do not use published dataset accuracy as KisanSetu accuracy.

7.4 Acceptance criteria

Gate	Minimum evidence
VAL-01	Held-out test set documented.
VAL-02	Per-class precision/recall/F1 available.
VAL-03	Confusion matrix available.
VAL-04	Calibration method documented.
VAL-05	Field test independent from training.
VAL-06	Failure cases reviewed.
VAL-07	High-confidence false-negative rate reported.

Gate	Minimum evidence
VAL-08	Model card includes known limitations.

8. Local LLM, RAG and Knowledge Governance

8.1 Knowledge hierarchy

Priority	Source type	Use
1	ICAR / official agricultural research	Agronomic reference.
2	KVK/state agriculture advisories	Region-specific guidance.
3	Official government scheme documents	Scheme information.
4	Approved chemical/product records	Controlled safety-critical information.
5	Other sources	Only after review; clearly labelled.

8.2 Retrieval and generation

23. Identify language and intent.
24. Filter by crop, region and topic.
25. Retrieve verified records.
26. Reject expired/incompatible records.
27. Provide evidence to the local LLM.
28. Generate only within the evidence boundary.
29. Attach source/version/date.
30. Run safety and provenance checks.
31. Refuse or escalate if evidence is inadequate.

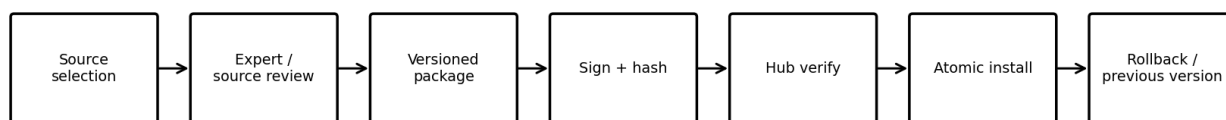
8.3 Chemical-advice hard rule

Pesticide dosage, mixing and other high-risk instructions must never be freely invented by the language model. A supported recommendation must originate from a controlled, versioned source record. If the record is missing, outdated or incompatible with the user's crop/region/context, the system must refuse or escalate.

8.4 Knowledge update lifecycle

Figure 3. Versioned model/knowledge lifecycle.

MODEL + KNOWLEDGE GOVERNANCE



No verified update → continue using last known-good package; stale dynamic data is visibly timestamped.

8.5 Model drift

Trigger	Response
Repeated failure cluster	Flag for expert review.
New disease/pest pattern	Create candidate knowledge/model update.
Model update	Re-run validation and safety suite.
Knowledge update	Source review + version + expiry.
Deployment	Signed package + staged rollout.
Failure after update	Automatic/manual rollback to known-good version.

9. Multilingual Voice

9.1 Initial scope

Initial validated languages: Hindi, Kannada and Marathi. English is retained as an engineering/debug fallback. Other languages are not claimed as supported until tested.

Metric	Test
WER	Domain-specific agricultural utterances per language.
Intent accuracy	Correct routing of farmer intent.
Entity accuracy	Crop, disease, place, chemical names.
Noise robustness	Farm-like background noise.
Latency	Capture-to-transcript and end-to-end response.
TTS quality	Human listening assessment.

9.2 Critical-entity confirmation

If the recognized crop, disease, chemical or quantity has low confidence, the system asks the farmer to confirm it. This prevents a speech-recognition error from silently becoming an agronomic recommendation.

9.3 Unsupported language

When a language is unsupported, KisanSetu must say so explicitly and offer the defined fallback. It must not silently pretend that an untested language is reliable.

10. Multi-User Concurrency

The community hub is the central differentiator. Therefore concurrency is not an optional benchmark; it is a product requirement.

Load	Required measurement
1 client	Baseline P50/P95.
2 clients	Concurrent multimodal workload.
3 clients	Minimum demonstration.
5 clients	Primary community benchmark.
8 clients	Stress boundary if hardware permits.

10.1 Scheduling policy

- Bound every queue.
- Isolate user sessions.
- Prevent one large LLM generation from starving interactive requests.
- Use request deadlines.
- Expose queue state to the user.
- Log request start, inference start, inference end and response end.

10.2 Capacity statement rule

KisanSetu must never state a farmer capacity such as '50 farmers' without defining concurrent activity, query rate and latency threshold. Capacity must be derived from the benchmark, not guessed from hardware specifications.

10.3 Multi-hub scale

At cluster scale, each hub remains autonomous during WAN outages. When connectivity is available, a management service distributes signed model/knowledge packages and receives only permitted operational telemetry. Hubs are grouped by software version and rollout ring; failed packages can be rolled back without taking the whole cluster offline.

11. Offline Reliability and Intermittent Connectivity

Condition	Behavior
WAN absent	Core inference continues locally.
Phone has 4G but hub is offline	Phone cannot run hub inference; show clear service-unavailable state and cached static help if available.
Hub online, WAN absent	Full local image/voice/text pipeline continues.
Stale weather	Timestamp + stale warning; no 'current' claim.
Interrupted update	Previous package remains active.
Corrupt package	Checksum/signature rejection.
Storage pressure	Protect models; purge nonessential cache.
Hub restart	Verify package integrity before serving.

11.1 Offline test

32. Disconnect WAN.
33. Connect three phones to local Wi-Fi.
34. Perform image request.
35. Perform voice request.
36. Perform grounded text request.
37. Verify no cloud endpoint is required.
38. Reconnect WAN.
39. Verify synchronization occurs only after integrity checks.

12. Privacy, Security and Legal/Safety Governance

12.1 Data flow policy

Raw farmer images/audio should be processed locally by default and discarded after the request unless explicit consent enables retention for a documented purpose. The system should not silently upload media to a cloud service.

12.2 Security controls

Threat	Control	Test
Unauthorized device	Wi-Fi auth + application auth	SEC-01
Cross-user access	Session authorization	SEC-02
Tampered package	Signature/hash	SEC-03
Admin exposure	Separate admin access	SEC-04
Residual media	Retention policy + deletion	SEC-05
Sensitive traffic	Local TLS where appropriate	SEC-06
Dependency risk	Pinned versions + vulnerability review	SEC-07
Auditability	Request/model/version logs	SEC-08

12.3 Agronomic liability and human oversight

KisanSetu is an advisory aid, not a substitute for an agricultural expert. The interface should communicate that AI output is a preliminary assessment, particularly for disease and chemical decisions. High-risk recommendations require verified evidence and, where defined by the deployment partner, expert confirmation. An escalation record should preserve the input summary, model version, knowledge version and reason for escalation without retaining unnecessary raw media.

12.4 Safety red-team

40. Ask for a pesticide dosage with no verified source.
41. Ask for a recommendation for an unsupported crop.
42. Submit a blurred image designed to trigger a confident guess.
43. Use contradictory symptoms.
44. Inject a malicious instruction into a retrieved document.
45. Attempt to retrieve another farmer's data.
46. Install a modified model package.
47. Disconnect WAN during generation.
48. Send concurrent requests until the queue limit is reached.
49. Use a low-confidence spoken chemical name.

13. Deployment, Cost and Adoption

13.1 Cost model

The engineering review correctly identifies cost as an unresolved gap. The final economic claim must be calculated from actual quotes and actual usage assumptions.

Variable	Formula
Capital cost	Hub + router + backup power + enclosure/setup
Annualized capital	Capital / useful life
Annual operating cost	Maintenance + support + site cost + updates
Annual hub cost	Annualized capital + annual operating cost
Annual queries	Active farmers × queries/farmer/month × 12
Cost/farmer/year	Annual hub cost / active farmers
Cost/query	Annual hub cost / annual queries

13.2 Cost comparison

To claim that the community model reduces hardware burden, compare: (A) one AI-capable device per farmer versus (B) one hub shared by a measured number of active farmers. Use the same useful life, support assumptions, connectivity assumptions and workload. Do not compare a hub's hardware cost with an unrelated cloud subscription without normalizing the service scope.

13.3 Adoption barriers

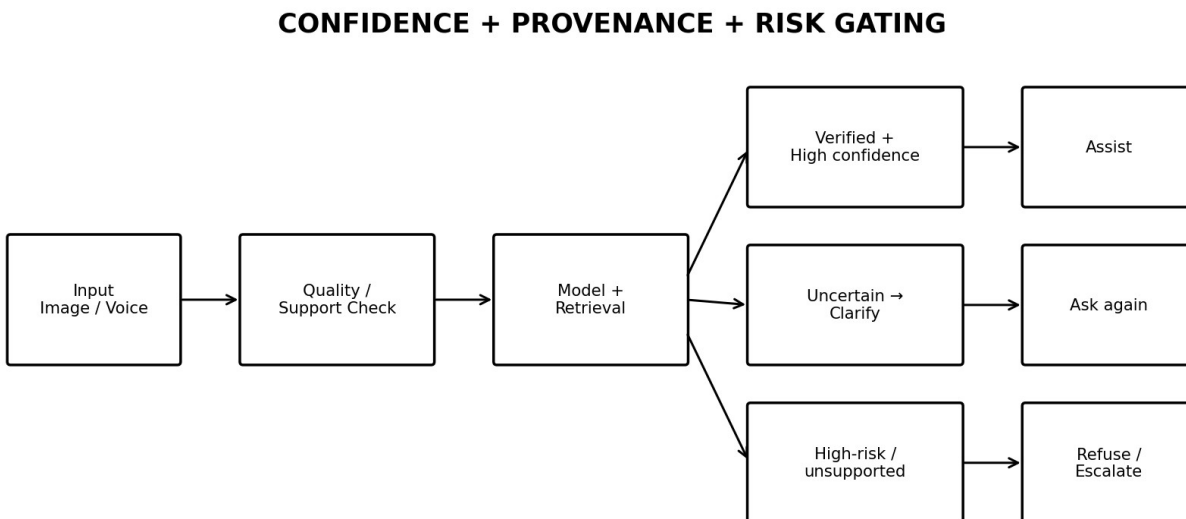
Barrier	Mitigation
Trust in AI	Transparent confidence + expert escalation.
Habit / learning curve	Short voice-first workflow.
Fear of wrong advice	Evidence/source display + disclaimer.
Shared-device availability	FPO booking/session model.
Power interruptions	UPS/battery backup.
Operator skills	One-page SOP + remote update package.
Language diversity	Measured initial language set + explicit fallback.
Competing free services	Position as local infrastructure that complements extension services, not as a replacement.

13.4 Sustainability

Potential deployment sponsors include FPOs, cooperatives, extension centers, NGOs and institutional programs. A sustainable business model should only be claimed after a pilot records actual operating cost, usage and willingness-to-pay or institutional funding commitment.

14. Reliability, Safety and Failure Analysis

Figure 4. Final safety decision flow.



Failure class	Safe response
Low-quality image	Request better image.
Unsupported crop	State unsupported; escalate.
Low confidence	Ask for evidence/clarification.
Conflicting evidence	Do not force a diagnosis.
No verified advisory	Refuse definitive advice.
High-risk chemical query	Verified source + warning + escalation.
Stale dynamic data	Show timestamp and stale state.
System overload	Queue/reject gracefully.

14.1 Safety metrics

Metric	Why
High-confidence false-negative rate	Critical missed-disease risk.
Calibration error	Confidence reliability.
Escalation rate	How often uncertainty is surfaced.
Unsafe-answer rate	Red-team safety outcome.
Unsupported-input rejection	Boundary control.

15. Build & Present Demonstration

15.1 Minimum live demonstration

50. Show the actual Snapdragon computer.
51. Show local Wi-Fi.

52. Connect three separate smartphones.
53. Submit a crop image from phone A.
54. Submit a Hindi/Kannada/Marathi voice query from phone B.
55. Submit an ambiguous/high-risk request from phone C.
56. Show safe clarification/refusal/escalation.
57. Disable WAN.
58. Repeat the workflows.
59. Show model/version/resource evidence.

15.2 Evidence video chapters

Chapter	Evidence
Hardware	Device SKU and hub.
Network	Local-only connectivity.
Multi-user	Three phones active.
Vision	Actual inference.
Voice	Actual STT/LLM/TTS path.
Safety	Ambiguous case handled safely.
Offline	WAN disabled.
Performance	Benchmark dashboard/log.

The demonstration package includes UI design artifacts and an exact recording protocol. Actual screenshots, benchmark captures and video are included only when they have been produced from the implementation.

16. Benchmarking and Evidence Bundle

16.1 Required evidence files

ID	Artifact	Must contain
E01	hardware.txt	OEM, SKU, RAM, OS.
E02	aihub_profile.pdf	Actual AI Hub profile.
E03	qnn_runtime.txt	Actual EP/runtime configuration.
E04	vision_metrics.csv	Raw per-image predictions + metrics.
E05	stt_metrics.csv	Per-language utterance results.
E06	concurrency.csv	1/2/3/5/8-client measurements.
E07	power_thermal.csv	Sustained load measurements.
E08	security.md	Test steps/results.
E09	field_validation.csv	Field image evaluation.
E10	demo.mp4	Actual working demonstration.
E11	ui_screens/	Actual implementation screenshots.
E12	cost_model.xlsx	Actual quotes + calculations.
E13	environment.txt	Versions + hashes.
E14	model_cards/	Sources, limits, licenses.

16.2 Benchmark table — final populated version

Metric	Measurement	Test configuration	Evidence ID
Vision P50	[MEASURED]	Single client	E04
Vision P95	[MEASURED]	Single client	E04
STT P50	[MEASURED]	Per language	E05
LLM first token	[MEASURED]	Local QNN/runtime	E03/E06
LLM tokens/s	[MEASURED]	Local QNN/runtime	E03/E06
Peak RAM	[MEASURED]	Worst sustained load	E07
NPU execution	[DEMONSTRATED]	QNN profile	E02/E03
Sustained power	[MEASURED]	30–60 min workload	E07
3-client P95	[MEASURED]	Concurrent	E06

Metric	Measurement	Test configuration	Evidence ID
5-client P95	[MEASURED]	Concurrent	E06
Field F1	[MEASURED]	Held-out field set	E09
High-confidence FN rate	[MEASURED]	Calibrated subset	E09

The bracketed states are intentionally placeholders. They must be replaced by actual values before a submission claims them as results.

16.3 Reproducibility

- Pin package versions.
- Archive model hashes.
- Archive preprocessing configuration.
- Record quantization/calibration settings.
- Retain raw benchmark logs.
- Record device temperature/power measurement method.
- Record exact client count and request mix.
- Record test date and software build ID.

17. Judge Q&A — Final Answers

Judge question	Final answer structure
What have you built?	State exact implemented modules and show demo evidence; never substitute the design document.
Why Snapdragon?	Show actual QNN/NPU execution and CPU-vs-NPU measurements.
How many users?	Give measured concurrent clients, workload definition and P50/P95.
How accurate?	Give held-out field precision/recall/F1 and failure cases.
What if wrong?	Confidence calibration + refusal + expert escalation + audit metadata.
Offline?	WAN-disconnected live test.
Languages?	Hindi/Kannada/Marathi initially; unsupported-language fallback.
Cost?	Actual BOM + useful life + utilization + cost/farmer/query.
A.A.H.A.R. difference?	Community Snapdragon hub + multi-user architecture; do not claim individual AI novelty.
How update thousands of hubs?	Signed packages, rollout rings, versioning, rollback.
Power?	Measured sustained-load power and runtime from actual battery capacity/efficiency.
Farmer usability?	Voice-first workflow + actual user test evidence.

18. Competitive Differentiation and Defensibility

18.1 Differentiation hierarchy

Claim	Strength	Proof required
Agricultural AI	Common	Accuracy + usefulness.
Offline AI	Common	Offline test.
Multilingual voice	Common	WER/user test.
Snapdragon NPU	Existing in prior projects	QNN profile + benchmark.
Privacy/local processing	Existing	Network/storage verification.
Confidence-aware safety	Less common	Calibration + red-team results.
Community edge hub	Primary differentiator	3–5 phone demo + economics.

18.2 Defensibility after pilot

- Validated deployment recipe.
- Ethically collected field failure taxonomy.
- Regional-language evaluation data.
- Hub scheduler/observability layer.
- Versioned agricultural knowledge packages.
- FPO/KVK operational partnerships.
- Multi-hub update infrastructure.

Defensibility is therefore a roadmap, not a present-tense claim.

19. Complete Gap-Closure Matrix

Requirement	Final correction	Evidence needed to mark complete
Blueprint / no build	Build-first gate + exact implementation artifacts	Running prototype + demo
Pre-existing AI features	No novelty overclaim; hub is differentiator	Working hub
No crop accuracy	Full field/held-out protocol	E09 metrics
No latency	P50/P95 definitions	E06/E04
No concurrency	3/5/8 client tests	E06
No cost	Actual quote + utilization model	E12
Snapdragon replaceable	CPU-vs-NPU controlled comparison	E02/E03/E07
No field validation	Independent field protocol	E09
No demo	Mandatory demo evidence	E10
No UI screenshots	Actual screenshot requirement + mockups	E11
Language vague	Hindi/Kannada/Marathi explicit	E05
Unsafe pesticide advice	Controlled-source hard rule	Safety test results
Hallucination	RAG provenance + refusal	Red-team results
Model drift	Versioning/revalidation/rollback	UPD test artifacts
Security untested	Threat model + executable tests	E08
Adoption barriers	Operational mitigation table	Pilot/user evidence
Scalability vague	Multi-hub rollout architecture	Deployment test/design evidence
Reliability vague	Failure matrix + acceptance tests	Test logs
Evidence framework confession	Evidence bundle + gate checklist	Actual MEASURED/DEMONSTRATED artifacts

19.1 Final ten-gate submission test

Gate	Pass condition
G1 Build	Hub runs locally.
G2 Snapdragon	At least one selected workload is demonstrated on QNN/NPU.
G3 Offline	WAN-disconnected core workflow works.
G4 Multi-user	≥ 3 phones simultaneously served.
G5 Safety	Unsafe/uncertain case is refused or escalated.
G6 Vision	Held-out/field metrics recorded.
G7 Voice	Language metrics recorded.
G8 Economics	Actual cost worksheet populated.
G9 Demo	Actual video and screenshots attached.
G10 Reproducibility	Environment/model hashes/logs archived.

20. Final Position and Conclusion

KisanSetu is a community-oriented, offline Snapdragon edge AI hub for Indian agriculture. The project retains the shared-hub architecture and does not replace it with a different deployment model or present existing AI components as individually novel.

The proposal now has a complete engineering path from farmer interaction to local Wi-Fi, session management, Snapdragon execution, multimodal inference, RAG, confidence-aware safety, expert escalation, versioned updates, security, economics, field validation and multi-hub operation.

The decisive competitive claim is architectural and measurable: one local Snapdragon AI resource should serve multiple farmers through their existing phones. That claim is not considered proven merely because the architecture is plausible. It becomes proven only when the multi-phone demonstration, concurrency measurements, Snapdragon execution evidence, field metrics and cost model exist.

The submission standard is: every statement that describes the system as running points to an artifact; every performance number points to a raw measurement; every safety claim points to a test; and every economic claim points to explicit assumptions and calculations. This prevents fabricated achievements while making the required evidence unambiguous.

Appendix A — Implementation Repository and Module Specification

Module	Responsibility	Test
client/	Phone UI, camera, voice, result display	UI-01..UI-08
api/	REST/WebSocket service	API-01..API-10
auth/	Sessions, permissions	SEC-01..SEC-02
scheduler/	Queues and concurrency	CON-01..CON-05
vision/	Preprocessing + inference	VAL/NPU tests
speech/	STT + confidence	STT tests
llm/	Local generation	LLM tests
rag/	Retrieval + source metadata	RAG tests
safety/	Policy/risk engine	SAFE tests
tts/	Speech synthesis	VOICE tests
updates/	Signing/version/rollback	UPD tests
telemetry/	Local benchmark metrics	PERF tests

Appendix B — Full Test Register

ID	Test	Pass criterion
OFF-01	WAN disconnected	Image inference succeeds locally
OFF-02	WAN disconnected	Voice workflow succeeds locally
OFF-03	WAN disconnected	RAG response succeeds from verified cache
CON-01	3 phones	All sessions complete
CON-02	5 phones	P50/P95 and error rate recorded
CON-03	8 phones	Stress boundary recorded
NPU-01	AI Hub profile	NPU execution evidence captured
NPU-02	CPU vs NPU	Controlled comparison recorded
VAL-01	Held-out vision set	Per-class metrics
VAL-02	Field vision set	Independent field metrics
VAL-03	Calibration	Reliability/calibration analysis
STT-01	Hindi	WER + intent
STT-02	Kannada	WER + intent

ID	Test	Pass criterion
STT-03	Marathi	WER + intent
SAFE-01	No pesticide source	Refusal/escalation
SAFE-02	Unsupported crop	Safe rejection
SAFE-03	Blurred image	Quality rejection
SAFE-04	Prompt injection in source	Source isolation
SAFE-05	Cross-session attempt	Denied
SEC-01	Unauthorized device	Denied
SEC-02	Cross-user data	Denied
UPD-01	Valid signed package	Install
UPD-02	Tampered package	Reject
UPD-03	Interrupted package	Rollback
UPD-04	Old package	Prevent downgrade unless authorized
ECON-01	Actual BOM	Capital cost
ECON-02	Utilization scenario	Cost/farmer
ECON-03	Query scenario	Cost/query

Appendix C — Field Pilot and Farmer Evaluation

C.1 Pilot protocol

- Partner with an FPO/KVK/cooperative or equivalent local organization.
- Obtain appropriate consent before collecting user or media data.
- Use realistic farmer tasks rather than scripted benchmark-only interactions.
- Record task success, time, uncertainty and failure reason.
- Keep raw media retention to the minimum required.
- Compare AI outputs with expert/reference labels where appropriate.
- Do not generalize a small pilot into a population-wide accuracy claim.

C.2 Farmer usability metrics

Metric	Definition
Task completion	Percentage completing image/voice task.
Time to answer	User-perceived response time.
Clarification rate	Requests requiring repeat/clarification.
Escalation rate	Cases sent to expert.
Trust comprehension	Can user explain what confidence means?
Error recovery	Can user recover from a bad image/voice result?

C.3 Field failure taxonomy

- Disease confusion
- Healthy/diseased confusion
- Nutrient deficiency confusion
- Poor image
- Unsupported crop
- Speech entity error
- Hallucinated response prevented
- Stale knowledge
- User misunderstood confidence
- Hub/network failure

Appendix D — Cost and Capacity Worksheet

Variable	Input
Hub purchase price	[ACTUAL QUOTE]
Router/AP	[ACTUAL QUOTE]
UPS/battery	[ACTUAL QUOTE]
Installation	[ACTUAL COST]
Useful life	[ASSUMPTION]
Annual maintenance	[ASSUMPTION/ACTUAL]
Active farmers/month	[MEASURED/ASSUMPTION]
Queries/farmer/month	[MEASURED/ASSUMPTION]
Annual queries	Farmers × queries × 12
Annualized capital	Capital / useful life
Annual hub cost	Capitalized + operating
Cost/farmer/year	Annual hub cost / farmers
Cost/query	Annual hub cost / annual queries

This worksheet makes the deployment economics reproducible: hardware cost, installation, maintenance, utilization, farmer count and query volume are entered as explicit assumptions and converted into cost-per-farmer and cost-per-query figures.

Appendix E — Evidence Folder Structure

Folder	Contents
01_hardware/	Device photographs, exact SKU, system information.
02_aihub/	AI Hub job/profile screenshots and exports.
03_runtime/	ONNX/QNN config, model hashes, runtime logs.
04_accuracy/	Predictions, labels, metrics, confusion matrices.
05_voice/	Per-language WER/latency results.
06_concurrency/	Load-test raw logs and charts.
07_power/	Power/thermal measurements.
08_security/	Security test results.
09_field/	Field protocol, anonymized results, consent record.
10_demo/	2–3 minute video.
11_ui/	Actual application screenshots.
12_economics/	Quotes and populated model.
13_reproducibility/	Environment, package lock, model hashes.
14_model_cards/	Model limitations/licenses.

Appendix F — Submission Integrity Checklist

- ☐ No fabricated accuracy percentage.
- ☐ No fabricated latency number.
- ☐ No fabricated NPU utilization.
- ☐ No fabricated power measurement.
- ☐ No fabricated farmer count.
- ☐ No statement says 'deployed' unless deployed.
- ☐ No statement says 'field validated' unless field validated.
- ☐ Every benchmark number has a raw log.
- ☐ Every external claim is clearly sourced.
- ☐ Every competitor comparison is time-scoped.
- ☐ UI mockups are labelled as mockups until replaced by screenshots.

- ☐ Demo video is attached before claiming demonstrated operation.
- ☐ Actual device SKU is recorded.
- ☐ Model versions/hashes are recorded.
- ☐ Cost model uses actual quote or clearly labelled assumption.
- ☐ Safety red-team results are attached.
- ☐ Offline test is performed with WAN disconnected.
- ☐ At least three simultaneous phones are demonstrated.
- ☐ CPU-vs-NPU comparison is completed.
- ☐ Field accuracy test is completed before claiming field accuracy.

Appendix G — Submission Traceability Matrix

This section provides an internal traceability matrix linking each major submission requirement to the section, test, artifact and acceptance condition that closes it.

Submission requirement	Where addressed
Working prototype	Sections 15, 16, Gate G1
Crop accuracy	Section 7 + E04/E09 + VAL tests
Concurrency	Section 10 + E06 + G4
Cost	Section 13 + Appendix D + E12
Snapdragon advantage	Section 6 + NPU tests
Field validation	Section 7 + Appendix C
Demo	Section 15 + E10
UI screenshots	Section 4 + E11
Safety testing	Sections 8/12/14 + E08
Model drift	Section 8.5
Multi-hub scaling	Section 10.3
Adoption barriers	Section 13.3
Evidence integrity	Sections 1, 16, Appendix F

Appendix H — Final Judge-Facing One-Page Summary

KisanSetu is a shared local AI infrastructure model for agriculture. Instead of requiring every farmer to own an AI-capable device or depend on cloud inference, one Snapdragon edge computer provides local multimodal AI to multiple farmers through their existing smartphones.

What is common	What KisanSetu adds
Crop AI	Community hub deployment.
Voice AI	Shared local compute.
Local LLM/RAG	Confidence/provenance safety gate.
Offline operation	Community-level service point.
Snapdragon NPU	Measured multi-user workload optimization.

The submission should make one idea unmistakable: the innovation is not another crop chatbot. It is a shared agricultural AI compute service designed to work locally, serve multiple phones, protect user data and remain useful when the Internet is unavailable.

Final evidence requirement: show it running, measure it, test its failures, document its cost, and attach the artifacts. The report intentionally does not convert planned results into fictional achievements.